

A nivel de ingeniería, este es uno de los sensores más robustos y complejos. El Bluetooth clásico (el de transferir archivos pesados o audio) ya no es el estándar para el ecosistema de Internet de las Cosas (IoT). Hoy en día, la industria se mueve sobre **BLE**, el cual opera bajo una arquitectura de **Cliente/Servidor** muy específica basada en perfiles, servicios y características.

Para este tutorial utilizaremos el paquete `flutter_blue_plus`, que es la biblioteca más estable, mantenida y utilizada a nivel profesional para interactuar con las antenas Bluetooth en iOS y Android de forma reactiva.

Tutorial: Redes Inalámbricas de Corto Alcance y Arquitectura Bluetooth Low Energy (BLE)

Este módulo práctico enseña los fundamentos de la comunicación por proximidad utilizando el estándar BLE, el escaneo asíncrono de periféricos en el entorno y la interacción con arquitecturas basadas en atributos de hardware.

1. Fundamentos Teóricos y Conceptos del Paquete `flutter_blue_plus`

El desarrollo profesional con Bluetooth moderno difiere completamente del emparejamiento clásico del sistema operativo. BLE está diseñado para transmitir ráfagas muy pequeñas de datos con un consumo energético mínimo.

- **Arquitectura Central/Periférico (Central vs Peripheral):** El teléfono móvil casi siempre actúa como el nodo **Central**. Es el encargado de iniciar el escaneo, buscar anuncios en el aire y solicitar conexiones. El dispositivo externo (un smartwatch, un sensor cardíaco o un ESP32) es el **Periférico**, que anuncia su presencia enviando paquetes de publicidad (*Advertising packets*).
- **Estructura GATT (Generic Attribute Profile):** Una vez que el teléfono se conecta a un periférico, la información no se envía en un canal plano, sino que se organiza en una estructura jerárquica:
- **Servicio (BluetoothService):** Una colección de datos lógicamente relacionados (por ejemplo, el "Servicio de Batería" o el "Servicio de Ritmo Cardíaco"). Cada servicio se identifica por un código único global llamado **UUID**.
- **Característica (BluetoothCharacteristic):** Se encuentra dentro de un servicio y es el punto final de datos real (el "sensor" específico). Una característica contiene el valor del dato y define sus propiedades de acceso: **Read** (Leer), **Write** (Escribir) o **Notify** (El periférico avisa reactivamente a Flutter cuando el dato cambia).
- **ScanResult (Resultado de Escaneo):** Objeto asíncrono emitido por el sensor de Bluetooth durante el rastreo. Contiene la identidad del periférico (`BluetoothDevice`), su dirección MAC o identificador interno, y la potencia de la señal medida en **RSSI** (un valor en dBm donde -50 es una señal excelente y -100 es una señal muy débil por distancia).

2. Configuración del Entorno (Plataforma Nativa)

El Bluetooth está estrechamente ligado a la privacidad porque puede usarse para rastrear la ubicación del usuario sin su consentimiento (mediante balizas o *Beacons*). Por ello, los sistemas operativos exigen permisos estrictos.

En Android (`android/app/src/main/AndroidManifest.xml`)

A partir de Android 12+, se requieren permisos específicos que separan el escaneo de la conexión. Añade estas líneas justo antes de la etiqueta `<application>`:

```
<uses-permission android:name="android.permission.BLUETOOTH_SCAN"
                android:usesPermissionFlags="neverForLocation" />
<uses-permission android:name="android.permission.BLUETOOTH_CONNECT" />
<uses-permission android:name="android.permission.BLUETOOTH"
                android:maxSdkVersion="30" />
<uses-permission android:name="android.permission.BLUETOOTH_ADMIN"
                android:maxSdkVersion="30" />
<uses-permission android:name="android.permission.ACCESS_FINE_LOCATION" />
```

En iOS (`ios/Runner/Info.plist`)

Añade las llaves que le explicarán por qué la aplicación encenderá el radio de Bluetooth:

```
<key>NSBluetoothAlwaysUsageDescription</key>
<string>Necesitamos acceso al Bluetooth para escanear periféricos en la práctica
de laboratorio.</string>
<key>NSBluetoothPeripheralUsageDescription</key>
<string>Necesitamos acceso al Bluetooth para interactuar con dispositivos
cercanos.</string>
```

Estructura de Carpetas del Módulo

Agregamos la nueva característica (`feature`) de forma ordenada en nuestro proyecto base:

```
mi_app_sensores/
├── pubspec.yaml
├── lib/
│   ├── main.dart
│   ├── core/
│   │   └── theme/
│   │       └── theme.dart
│   └── features/
│       ├── home/
│       ├── bluetooth/
│       │   └── screens/
│       └── <-- CARPETA DEL MÓDULO ACTUAL
```

└─ bluetooth_screen.dart

Asegúrate de agregar la dependencia oficial en el archivo *pubspec.yaml*:

```
dependencies:  
  flutter:  
    sdk: flutter  
  flutter_blue_plus: ^1.34.0 # 0 la versión estable más reciente
```

3. Código Fuente Completo y Comentado

Archivo *lib/features/bluetooth/screens/bluetooth_screen.dart*

```
import 'package:flutter/material.dart';  
import 'package:flutter_blue_plus/flutter_blue_plus.dart';  
import 'dart:async';  
import 'dart:io';  
  
class BluetoothScreen extends StatefulWidget {  
  const BluetoothScreen({super.key});  
  
  @override  
  State<BluetoothScreen> createState() => _BluetoothScreenState();  
}  
  
class _BluetoothScreenState extends State<BluetoothScreen> {  
  List<ScanResult> _scanResults = [];  
  bool _isScanning = false;  
  
  // Suscripciones a los Streams globales de la librería  
  StreamSubscription<List<ScanResult>>? _scanResultsSubscription;  
  StreamSubscription<bool>? _isScanningSubscription;  
  
  @override  
  void initState() {  
    super.initState();  
    _initBluetoothListeners();  
  }  
  
  // 1. Inicializar la escucha reactiva del adaptador de hardware  
  void _initBluetoothListeners() {  
    // Escuchar los resultados que el escáner va encontrando en el aire en tiempo  
    real  
    _scanResultsSubscription = FlutterBluePlus.scanResults.listen((results) {  
      setState(() {  
        _scanResults = results;  
      });  
    });  
  }  
}
```

```

    });
  });

  // Monitorear si el hardware sigue buscando o ya se detuvo el temporizador
  _isScanningSubscription = FlutterBluePlus.isScanning.listen((scanning) {
    setState(() {
      _isScanning = scanning;
    });
  });
}

// 2. Control del ciclo de Escaneo de Periféricos
Future<void> _toggleScan() async {
  // Validar primero si el Bluetooth físico del teléfono está encendido
  if (await FlutterBluePlus.adapterState.first != BluetoothAdapterState.on) {
    _showSnackBar("Por favor, enciende el Bluetooth de tu dispositivo.");
    return;
  }

  try {
    if (_isScanning) {
      // Detener el hardware de forma manual
      await FlutterBluePlus.stopScan();
    } else {
      // Limpiar la lista previa antes de iniciar un nuevo ciclo
      setState(() {
        _scanResults.clear();
      });

      // Iniciar el escaneo con un timeout de 15 segundos para no agotar la
      batería
      // (En ingeniería móvil, nunca se deja un escaneo de radio corriendo
      indefinidamente)
      await FlutterBluePlus.startScan(timeout: const Duration(seconds: 15));
    }
  } catch (e) {
    _showSnackBar("Error al interactuar con el hardware: $e");
  }
}

void _showSnackBar(String message) {
  ScaffoldMessenger.of(context).showSnackBar(SnackBar(content: Text(message)));
}

@override
void dispose() {
  // CRUCIAL: Detener el escaneo si el usuario sale abruptamente de la pantalla
  if (_isScanning) {
    FlutterBluePlus.stopScan();
  }
  // Cancelar las suscripciones para evitar fugas de memoria críticas
  _scanResultsSubscription?.cancel();
  _isScanningSubscription?.cancel();
  super.dispose();
}

```

```

}

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(title: const Text('Sensor: Redes Bluetooth BLE')),
    body: Padding(
      padding: const EdgeInsets.all(16.0),
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.stretch,
        children: [
          // Panel de Control del Escáner
          Card(
            elevation: 4,
            color: _isScanning ? Colors.blue.withOpacity(0.05) :
Colors.grey.withOpacity(0.05),
            child: Padding(
              padding: const EdgeInsets.all(16.0),
              child: Column(
                children: [
                  Text(
                    _isScanning ? '🔍 Escaneando el Entorno...' : 'Antena en
Espera',
                    style: const TextStyle(fontSize: 16, fontWeight:
FontWeight.bold),
                  ),
                  const SizedBox(height: 8),
                  const Text(
                    'Busca paquetes de publicidad (Advertising) emitidos por
dispositivos BLE cercanos.',
                    textAlign: TextAlign.center,
                    style: TextStyle(fontSize: 12, color: Colors.grey),
                  ),
                  const SizedBox(height: 12),
                  ElevatedButton.icon(
                    onPressed: _toggleScan,
                    style: ElevatedButton.styleFrom(
                      backgroundColor: _isScanning ? Colors.redAccent :
Colors.blueAccent,
                      foregroundColor: Colors.white,
                    ),
                    icon: Icon(_isScanning ? Icons.bluetooth_disabled :
Icons.bluetooth_searching),
                    label: Text(_isScanning ? 'Detener Búsqueda' : 'Iniciar
Escaneo BLE'),
                  ),
                ],
              ),
            ),
          const SizedBox(height: 12),
          const Text(
            'Dispositivos Detectados:',

```

```

        style: TextStyle(fontSize: 14, fontWeight: FontWeight.bold),
      ),
      const SizedBox(height: 8),

      // Lista Dinámica de Dispositivos Encontrados en el Aire
      Expanded(
        child: _scanResults.isEmpty
          ? Center(
              child: Text(
                _isScanning ? 'Buscando señales...' : 'Lista vacía. Inicia
un escaneo.',
                style: const TextStyle(color: Colors.grey),
              ),
            )
          : ListView.builder(
              itemCount: _scanResults.length,
              itemBuilder: (context, index) {
                final result = _scanResults[index];
                final deviceName =
result.advertisementData.advName.isNotEmpty
                  ? result.advertisementData.advName
                  : "Dispositivo no nombrado";

                return Card(
                  margin: const EdgeInsets.symmetric(vertical: 4),
                  child: ListTile(
                    leading: Icon(
                      Icons.devices,
                      color: result.rssi > -70 ? Colors.green :
Colors.orange,
                    ),
                    title: Text(deviceName, style: const
TextStyle(fontWeight: FontWeight.bold)),
                    subtitle: Text('ID: ${result.device.remoteId}\nRSSI:
${result.rssi} dBm'),
                    trailing: Text(
                      '${result.advertisementData.connectable ?
"Conectable" : "Solo Adv"}',
                      style: TextStyle(
                        fontSize: 11,
                        color: result.advertisementData.connectable ?
Colors.teal : Colors.grey,
                        fontWeight: FontWeight.bold
                      ),
                    ),
                  ),
                );
              },
            );
    ],
  ),
),
);

```

```
}  
}
```

Vinculación en el Menú Principal

Abre el archivo de control central `lib/features/home/screens/home_screen.dart` para enlazar la navegación real de la tarjeta de Bluetooth:

```
// 1. Agrega el import correspondiente al inicio del archivo:  
import '../..bluetooth/screens/bluetooth_screen.dart';  
  
// 2. Modifica la tarjeta de Bluetooth dentro del GridView:  
_SensorCard(  
  title: 'Bluetooth BLE',  
  icon: Icons.bluetooth,  
  color: Colors.blue, // Cambiamos de gris a azul indicando que está activo  
  onTap: () {  
    Navigator.push(  
      context,  
      MaterialPageRoute(builder: (context) => const BluetoothScreen()),  
    );  
  },  
),
```

Guía Didáctica (Puntos Clave para Explicar en Clase)

1. **La Importancia del RSSI (Métricas de Señal):** El valor `result.rssi` es un indicador directo de distancia física. Dejen la app escaneando y alejen o acerquen otro teléfono con el Bluetooth encendido; verán cómo el número fluctúa. Esto les sirve para implementar lógicas basadas en proximidad (como abrir una puerta inteligente solo si el RSSI es mayor a -60 dBm).
 2. **Políticas de Ahorro de Energía (Timeout):** El parámetro `timeout: const Duration(seconds: 15)`. En sistemas de ingeniería, dejar un chip de radio buscando señales indefinidamente consume la batería de forma crítica y satura los hilos de comunicación del sistema operativo.
 3. **Filtrado por Nombre Vacío:** Muchos dispositivos aparecen como "Dispositivo no nombrado". Muchas balizas o periféricos de IoT no transmiten un String con su nombre comercial en el paquete publicitario básico para ahorrar bytes de transmisión; solo envían su ID de hardware y datos crudos.
-